Adaptive Learning Control over trials for Robust Trajectory Tracking in Robotic Systems

Jal Dani 202201315
Patel Divy 202201495
Mentor: Sujay Kadam

***Abstract*—This paper presents an adaptive learning control strategy for trajectory tracking in robotic systems subject to environmental perturbations. The proposed controller iteratively updates its control input based on errors from previous trials, enabling the robot to learn and adapt over time. The approach is validated on simulations where the robot is trained to follow various trajectories, including square, ellipse, and hexagon paths. Simulation results demonstrate robust performance, with reduced tracking errors and improved adaptation under perturbations. These findings suggest that the proposed method can enhance robot performance in uncertain environments.**

***Index Terms*—Adaptive control, Iterative learning control, Trajectory tracking, Robotic systems, Perturbation adaptation.**

I. Introduction

Robotic systems have been developed over the past several decades to perform tasks that are both repetitive in nature and require high precision. Traditional controller designs typically rely on accurate models of the robot's dynamics and kinematics. However, precise knowledge of these models is not always available in practical applications. This challenge has driven the exploration of alternative methods such as adaptive control, intelligent control techniques, and learning control approaches .

With the rapid evolution of self-learning capabilities in robotics, there is an increasing demand for controllers that can adapt to systematic changes in the environment particularly when the operating conditions vary over time. In contrast, human motor learning exhibits a high degree of sophistication, adapting to environmental perturbations on a trial-by-trial basis. Key features observed in human motor learning, such as *savings* (faster re-learning of a previously encountered perturbation) , *retention* (maintaining learned adaptations with reduced initial errors), and *generalization* (transfer of learned behavior to similar tasks), offer valuable insights for developing robust control algorithms for robots.

Motivated by these insights, this paper proposes an adaptive learning controller that iteratively updates its control actions based on errors from previous trials. The proposed approach integrates iterative learning control (ILC) with feedforward inversion-based control and internal model-based error prediction. This combination is aimed at enhancing trajectory tracking performance under external perturbations. To validate the approach, simulation studies are conducted on a robotic system tasked with following various trajectories including square, ellipse, and hexagon paths thus emulating real-world scenarios with diverse geometric challenges.

In summary, this work contributes a novel, simple-to-implement adaptive learning controller inspired by human motor learning principles, and demonstrates its effectiveness in robust trajectory tracking through extensive simulation studies.

II. Control Strategy

The proposed control strategy integrates several key components that work synergistically to enhance the performance and robustness of the learning controller. Each component addresses different aspects of the control problem, ensuring that the system can adapt to disturbances and uncertainties while tracking the desired trajectory.

A. Iterative Learning Control (ILC)

The ILC mechanism is designed to update the control input based on the error observed in previous trials. For a given trial k and time index t, the ILC update rule can be expressed as:

$$u_{\text{ILC},k}(t) = \lambda_{\text{ILC}}\, u_{\text{ILC},k-1}(t) + \gamma_{\text{ILC}}\, \tanh\Big(\varepsilon_{\text{TE},k-1}(t)\Big), \quad (1)$$

where:

- $\varepsilon_{\text{TE},k-1}(t) = y_{\text{ref},k-1}(t) - y_{k-1}(t)$ is the task error, defined as the difference between the reference trajectory and the actual output from the previous trial.
- $\lambda_{\text{ILC}} \in [0, 1]$ is a memory (or retention) factor that determines how much of the previous control input is retained.
- $\gamma_{\text{ILC}} \in [0, 1]$ is the learning rate that scales the correction based on the task error.

This process ensures that the control input is iteratively refined, allowing the system to progressively "learn" the appropriate control actions that minimize the tracking error over successive trials.

B. Feedforward Control

In parallel with the ILC, a feedforward control component is introduced to proactively counteract the system dynamics and anticipated disturbances. The feedforward term is computed using an inversion-based approach, typically involving the first nonzero Markov parameter M_ρ of the system. The corresponding update is given by:

$$u_{\text{FF},k}(t) = \lambda_{\text{FF}}\, u_{\text{FF},k-1}(t) + \gamma_{\text{FF}}\, M_\rho^\dagger\, \tanh\Big(\varepsilon_{\text{MOE},k-1}(t)\Big), \quad (2)$$

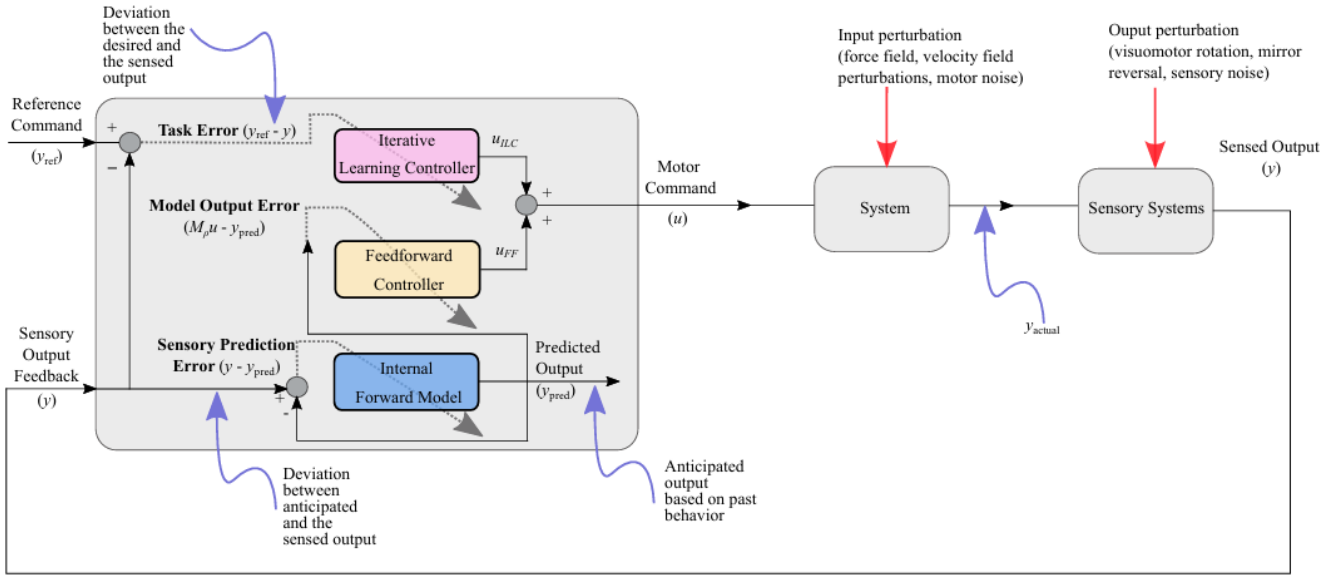

Fig. 1. A schematic block-diagrammatic representation of the proposed learning controller (2)– (5) with three interacting processes. The task error drives the direct update of one of the components (ILC component) of the input, while the sensory prediction error drives the update of the internal forward model representation. The model output error, that is, the difference between the expected outcome of motor commands and internal prediction of sensory output is used to modulate the feedforward action component of the input.

where:

- $M_\rho^\dagger$ is the Moore-Penrose pseudoinverse of M_ρ, representing an approximate inverse model of the system dynamics.
- $\varepsilon_{\text{MOE},k-1}(t) = M_\rho\, u_{k-1}(t) - y_{\text{pred},k-1}(t)$ is the model output error, indicating the discrepancy between the anticipated effect of the previous control input and the predicted output.
- λ_{FF} and γ_{FF} are analogous memory and learning rate factors for the feedforward term.

This feedforward control action helps to rapidly compensate for predictable disturbances, thereby reducing the burden on the reactive ILC component.

C. Predictive Component

The predictive component further enhances control performance by estimating the system's output response to the applied control input. This is achieved by updating a predicted sensory output, which is then compared with the actual output to generate a sensory prediction error:

$$\varepsilon_{\text{SPE},k-1}(t) = y_{k-1}(t) - y_{\text{pred},k-1}(t). \tag{3}$$

The predicted sensory output is updated as follows:

$$y_{\text{pred},k}(t) = \lambda_{\text{pred}}\, y_{\text{pred},k-1}(t) + \gamma_{\text{pred}}\, \tanh\Big(\varepsilon_{\text{SPE},k-1}(t)\Big), \tag{4}$$

where λ_{pred} and γ_{pred} control the retention and learning rate for the predictive model. This component forms an internal representation of the system dynamics, allowing the controller to anticipate and mitigate future errors, especially those arising from delays or unexpected disturbances.

D. Combined Control Input

The total control input applied to the system is the sum of the ILC and feedforward components:

$$u_k(t) = u_{\text{ILC},k}(t) + u_{\text{FF},k}(t). \tag{5}$$

By integrating these three components—reactive learning through ILC, proactive disturbance compensation via feedforward control, and anticipatory adjustments through the predictive model—the controller achieves robust performance, adapting effectively to both known and unforeseen perturbations over successive trials.

III. Simulation Setup for the 2R Robot

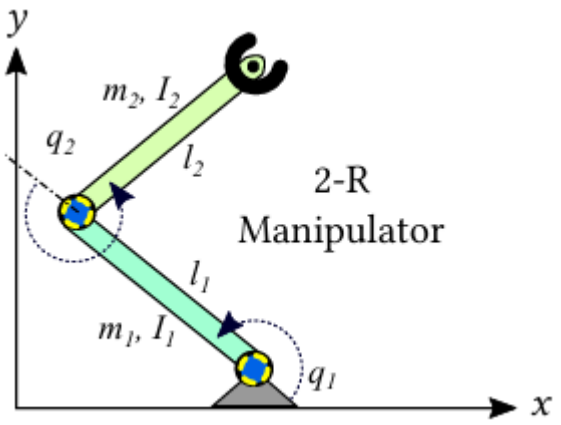

Fig. 2. Simplified 2R planar elbow manipulator.

In this study, a simplified 2R planar elbow manipulator is employed to evaluate the performance of the proposed learning controller. The robot model serves as a testbed for analyzing how the controller adapts under different dynamic representations and perturbation conditions. Two distinct simulation scenarios are considered:

A. Linearized Dynamics Simulation

In the first simulation, the 2R robot is modeled using linearized dynamics. In this representation:

- **Actuation and Perturbations:** The control input and external perturbations are applied directly at the joints. This simplifies the dynamics, making it easier to isolate and study the impact of the controller on joint variables.
- **Trajectory Tracking:** Desired trajectories, such as square, ellipse, and hexagon paths, are defined in joint space. Tracking errors are computed as the difference between the reference joint angles and the actual joint angles produced by the robot.

This setup allows for a detailed examination of how the iterative learning control (ILC) and feedforward components contribute to reducing tracking errors in a controlled, linear environment.

B. Nonlinear Kinematics Simulation

The second simulation introduces a more realistic representation of the robot by incorporating nonlinear kinematics:

- **Actuation:** Similar to the linear simulation, control inputs are applied at the joints.
- **End-Effector Task Space:** In contrast, the desired trajectories, errors, and applied perturbations are now defined at the robot's end effector. This introduces nonlinear mapping between joint angles and the end-effector position.
- **Error Definition:** The tracking error is measured as the deviation between the desired and actual positions of the end effector. This requires an additional layer of transformation from joint space to Cartesian space.

By comparing the two simulations, the robustness and adaptability of the proposed controller are evaluated under both simplified and complex, real-world conditions.

C. Simulation Protocol and Perturbation Paradigm

To emulate human-like learning properties, the simulation protocol follows a sequential trial structure that mirrors paradigms observed in human motor adaptation:

1) **Baseline Trials:** Initial trials are conducted without any perturbations to establish a reference performance. These trials allow the controller to learn the nominal dynamics of the robot.
2) **Perturbation Trials (PT1):** A planned perturbation is introduced to the system. In the linearized dynamics simulation, this perturbation is applied as a constant external joint torque, while in the nonlinear simulation, a constant cross force field is applied at the end effector. This phase tests the controller's capability to adapt to unexpected changes.
3) **Washout Trials:** After the initial adaptation phase, a series of washout trials are performed where the perturbation is removed. This phase assesses the retention of learned adaptations and the ability of the controller to revert to baseline performance.
4) **Re-Perturbation Trials (PT2):** Finally, another set of perturbation trials is conducted. The response during this phase is used to evaluate the phenomenon of *savings*—where the controller exhibits faster re-learning of the perturbation due to retained adaptive features.

D. Key Performance Metrics

Throughout the simulation, the following metrics are used to assess controller performance:

- **Tracking Error:** Quantified as the norm of the difference between the desired and actual positions (or joint angles) across the trial.
- **Adaptation Speed:** Measured by the rate at which tracking errors decrease over successive trials.
- **Robustness:** Evaluated based on the controller's ability to maintain performance in the presence of external disturbances.

Overall, this simulation setup provides a comprehensive environment to validate the proposed learning controller's effectiveness in achieving robust trajectory tracking and adaptation in robotic systems.

IV. Trajectory Generation

For validation, three types of trajectories are generated using a minimum-jerk profile to ensure smooth motion. In general, the time-scaling function is defined as

$$s(\tau) = 10\tau^3 - 15\tau^4 + 6\tau^5, \quad \tau = \frac{t_{\text{seg}}}{T_{\text{seg}}} \in [0,1], \tag{6}$$

where t_{seg} is the elapsed time in the current segment and T_{seg} is the duration of that segment.

A. Square Trajectory

The square trajectory is constructed by dividing the motion into four segments corresponding to the four edges. Let the bottom-left corner be (x_i, y_i) and the top-right corner be (x_f, y_f). Then, for each segment:

- **Bottom edge (moving right):**

$$x(t) = x_i + (x_f - x_i)\, s(\tau), \quad y(t) = y_i. \tag{7}$$

- **Right edge (moving up):**

$$x(t) = x_f, \quad y(t) = y_i + (y_f - y_i)\, s(\tau). \tag{8}$$

- **Top edge (moving left):**

$$x(t) = x_f - (x_f - x_i)\, s(\tau), \quad y(t) = y_f. \tag{9}$$

- **Left edge (moving down):**

$$x(t) = x_i, \quad y(t) = y_f - (y_f - y_i)\, s(\tau). \tag{10}$$

In addition to the spatial parameterization, time stamps for switching between segments are computed to ensure continuity in velocity and acceleration, taking advantage of the smooth nature of the minimum-jerk profile.

B. Ellipse Trajectory

For the ellipse, with center (x_c, y_c) and semi-axes a (semi-major) and b (semi-minor), the trajectory is parameterized by an angle that evolves smoothly from a given starting angle θ_0. The evolution is defined by:

$$\theta(t) = \theta_0 + 2\pi\, s(\tau), \tag{11}$$

and the position on the ellipse is given by

$$x(t) = x_c + a\cos\theta(t), \quad y(t) = y_c + b\sin\theta(t). \tag{12}$$

This formulation allows for an adjustable starting phase θ_0 which can be randomized or set according to specific task requirements. The minimum-jerk profile ensures that the angular velocity changes smoothly over time.

C. Hexagon Trajectory

The hexagon trajectory is constructed by first determining the vertices of a regular hexagon. Let the hexagon have a side length L and be centered at (x_c, y_c). The vertices are calculated as:

$$x_{\text{hex},i} = x_c + L\cos\theta_i, \quad y_{\text{hex},i} = y_c + L\sin\theta_i, \tag{13}$$

with

$$\theta_i = \frac{2\pi(i-1)}{6}, \quad i = 1, \ldots, 7, \tag{14}$$

where the 7th vertex repeats the first vertex to close the hexagon. For each segment connecting vertex i to vertex $i+1$, the position is obtained by linear interpolation using the minimum-jerk profile:

$$x(t) = x_{\text{hex},i} + s(\tau)\Big(x_{\text{hex},i+1} - x_{\text{hex},i}\Big), \tag{15}$$

$$y(t) = y_{\text{hex},i} + s(\tau)\Big(y_{\text{hex},i+1} - y_{\text{hex},i}\Big). \tag{16}$$

This approach guarantees that the motion along each edge is smooth, and the transition between edges is continuous in position, velocity, and acceleration.

Thus, the trajectories for the square, ellipse and hexagon are generated by combining the spatial parameterization with the smooth time-scaling function given in Eq. (6), ensuring smooth transitions and minimizing abrupt changes.

V. Simulation Results

A. Square Trajectory Simulation Results

The simulation results for the square trajectory demonstrate that the controller accurately follows the desired piecewise-linear path. Sharp corners are handled effectively with minimal deviation, and the error metrics (TE, SPE, and MOE) consistently decrease over time, indicating robust model adaptation.

Figure 4. illustrates the evolution of the norm of individual components—task error (TE), sensory prediction error (SPE), and model output error (MOE)—over successive iterations. Initially, the TE decreases sharply as the controller begins to learn the system dynamics, although periodic spikes occur when perturbations are introduced. The SPE gradually reduces,

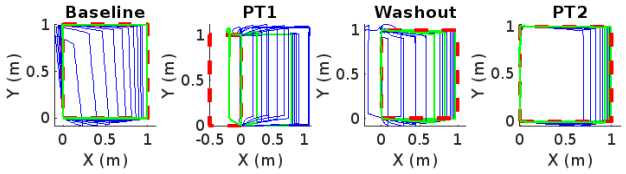

Fig. 3. Tracked square trajectory showing the system's ability to follow the reference path with minimal deviation.

indicating that the internal forward model is progressively improving its predictions of the motor outcomes. Meanwhile, the MOE stabilizes at a bounded level, reflecting the refinement of the controller's predictive mechanism.

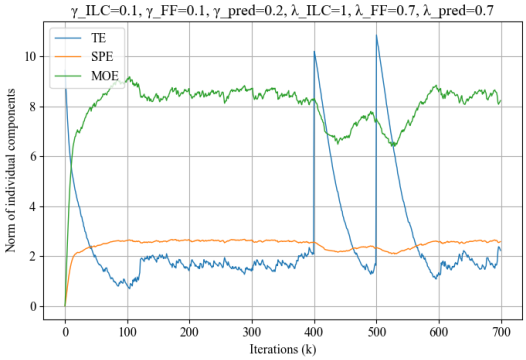

Fig. 4. Evolution of TE, SPE, and MOE during task execution.

Figure 5. presents a comparative analysis of the cumulative error across matched iterations for different phases: baseline, first perturbation (PT1), washout, and second perturbation (PT2). The baseline phase shows the lowest cumulative error, demonstrating the system's optimal performance under unperturbed conditions. In contrast, PT1 exhibits a significant increase in error, highlighting the initial adaptation challenge posed by the disturbance. During the washout phase, errors reduce yet remain elevated relative to baseline. Notably, the error during PT2 is considerably lower than that of PT1, confirming the phenomenon of *savings*—the controller relearns faster when encountering a familiar perturbation.

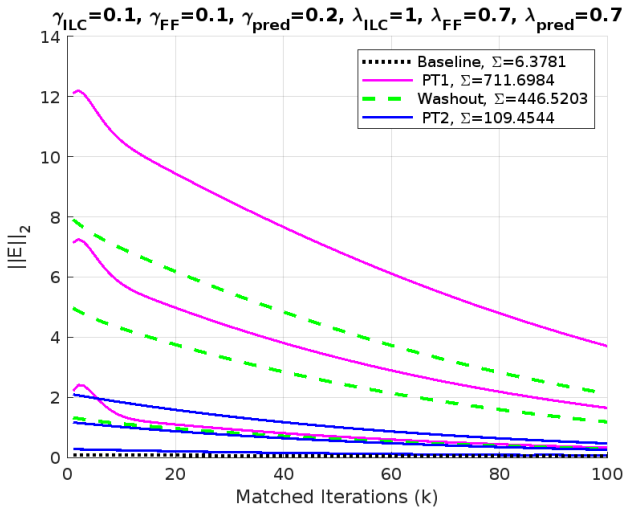

Fig. 5. Matched iterations error comparison across baseline, PT1, washout, and PT2 phases.

B. Ellipse Trajectory Simulation Results

The simulation results for the ellipse trajectory illustrate the controller's ability to accurately follow smooth elliptical paths. The system demonstrates excellent tracking performance with minimal deviation, and the error metrics (TE, SPE, and MOE) consistently decline over iterations, confirming robust adaptation. Furthermore, the cumulative error analysis across different phases—baseline, first perturbation (PT1), washout, and second perturbation (PT2)—highlights the phenomenon of *savings*, where the controller re-learns faster when facing a familiar disturbance.

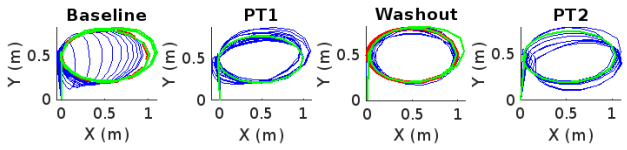

Fig. 6. Tracked ellipse trajectory showing the system's performance in following the smooth elliptical path.

Figure 6 shows that the system maintains a consistent and accurate path along the elliptical shape.

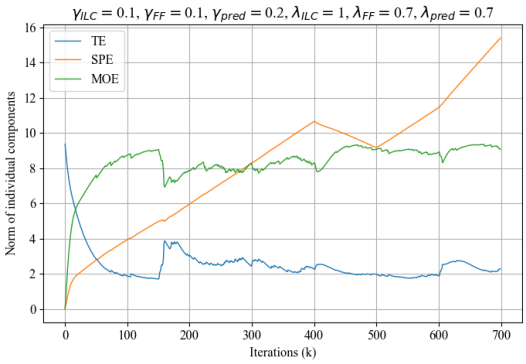

Fig. 7. Evolution of TE, SPE, and MOE during elliptical path tracking.

Figure 7 shows that the task error (TE) drops sharply during the early iterations, indicating a rapid improvement in tracking the desired trajectory. In contrast, the sensory prediction error (SPE) and model output error (MOE) decline more steadily, reflecting the gradual refinement of the internal model. Together, these trends highlight effective learning: the controller quickly minimizes performance errors while the predictive model adapts to accurately represent system dynamics.

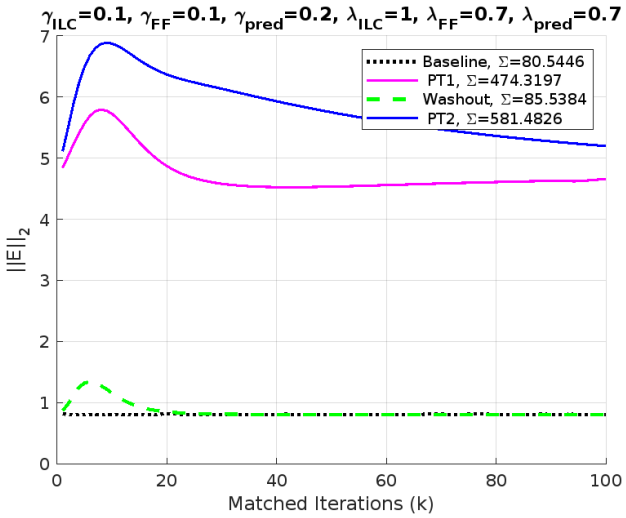

Fig. 8. Matched iterations error comparison across baseline, PT1, washout, and PT2 phases for the ellipse trajectory.

Figure 8 provides a comparative analysis of the cumulative error. The baseline phase shows the lowest error, whereas PT1 exhibits a significant spike. During the washout phase, errors partially recover, and the notably lower error in PT2 confirms that the controller benefits from *savings*, re-learning faster when encountering a similar perturbation.

C. Hexagon Trajectory Simulation Results

The simulation results for the hexagon trajectory reveal a noticeably poorer performance compared to the square and ellipse cases. The tracked trajectory (Figure 9) deviates significantly from the desired hexagon path, particularly near the corners, indicating inadequate compensation for dynamic uncertainties. In Figure 10, the task error (TE) not only fails to decrease but instead increases over successive iterations, while both the sensory prediction error (SPE) and model output error (MOE) remain persistently high. Finally,

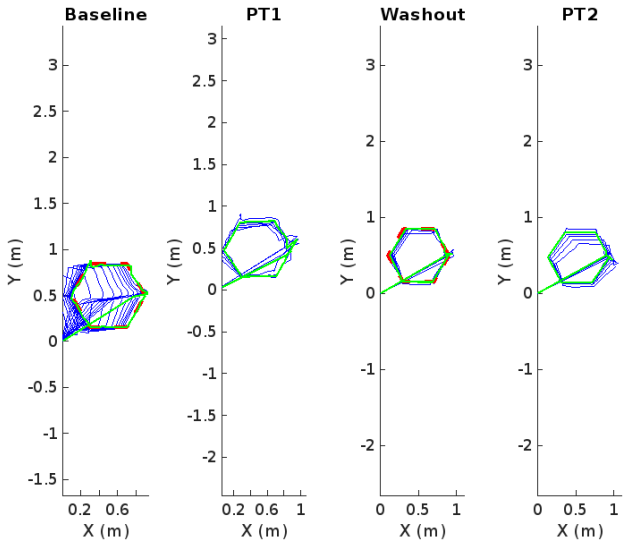

Fig. 9. Tracked hexagon trajectory showing significant deviation from the desired six-edge path.

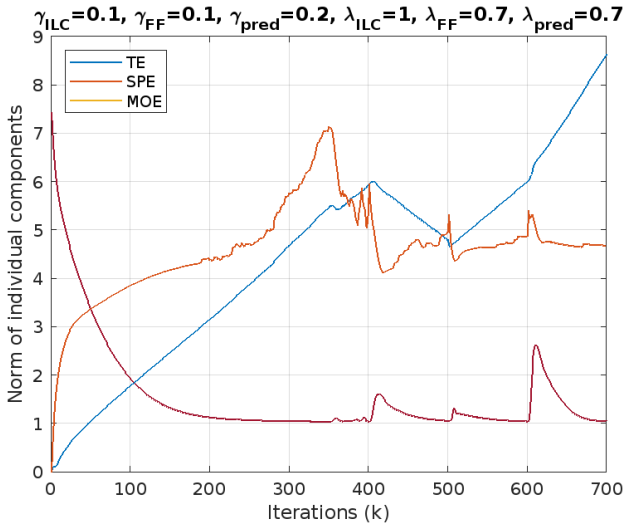

Fig. 10. Evolution of TE, SPE, and MOE during hexagon path tracking. Note the increasing trend in TE.

VI. Conclusion

An adaptive learning controller integrating iterative learning control (ILC) with feedforward and predictive mechanisms was presented for robotic trajectory tracking. The simulation results on square and ellipse trajectories, which serve as the protagonists, demonstrate that the controller can effectively reduce tracking errors and rapidly adapt to dynamic uncertainties and external disturbances. In these cases, the error signals converge satisfactorily, and the controller exhibits robust performance over successive iterations.

In contrast, the hexagon trajectory results reveal antagonistic behavior. The controller struggles with the more complex, multi-angular path, leading to significant deviations from the desired trajectory and an increasing trend in task error (TE) across iterations. This discrepancy highlights the limitations of the current controller configuration when faced with more challenging trajectory geometries.

Overall, while the proposed approach shows considerable promise for simpler trajectory shapes, the contrasting performance on the hexagon case underscores the need for further refinement. Future work will focus on experimental validation and the development of enhanced adaptive gain tuning strategies to bolster the controller's robustness, particularly for complex and non-smooth trajectory profiles.

Acknowledgments

We would like to express our sincere gratitude to Sujay Sir for his invaluable support throughout this project. His insightful ideas, deep understanding of the subject matter, and assistance with coding and literature have been instrumental in shaping the direction and success of our work. We are truly grateful for his guidance and encouragement.

VII. Code and Supplementary Materials

All simulation codes, figures, and other resources used in this project can be accessed at the following link:

Google Drive - Project Files

References

Kadam, S., Jadav, S., Mehta, A., & Palanthandalam-Madapusi, H. (2022). A Novel Learning Control Structure Exhibiting Features of Human Motor Learning. *IEEE Transactions on Cognitive and Developmental Systems*, SI: Emerging Topics on Development and Learning. Manuscript ID TCDS-2022-0069.R2.